

PENSAMENTO COMPUTACIONAL



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Pensamento lógico e algorítmico

Marcelo da Silva dos Santos

OBJETIVOS DE APRENDIZAGEM

- > Reconhecer o papel da lógica no pensamento computacional.
- > Definir pensamento algorítmico.
- > Descrever as dificuldades recorrentes com lógica e algoritmos.

Introdução

O termo **pensamento computacional** (do inglês, *computational thinking*) foi apresentado na década de 1980 por Seymour Papert e, em 2006, a pesquisadora Jeannette Wing publicou um artigo sobre o tema, que passou a receber atenção da comunidade científica e popularizou a ideia de utilizar a tecnologia como recurso interdisciplinar. A primeira impressão que se tem do tema é que está ligado ao uso da tecnologia, mas vai muito além disso, sendo fortemente relacionado ao estímulo ao raciocínio lógico e à criatividade.

O conceito visa ao desenvolvimento lógico e algorítmico para resolução de problemas não apenas da área da computação, mas de todas as áreas de conhecimento. Para isso, é de grande importância o estudo de dois outros conceitos: o pensamento lógico e o pensamento algorítmico. Ambas as áreas são habilidades que podem ser desenvolvidas por qualquer pessoa, e não somente por cientistas da computação e profissionais de tecnologia, e aplicadas nas atividades diárias, estimulando a criatividade e a produtividade por meio da lógica e da construção de algoritmos.

Neste capítulo, você vai conhecer os conceitos de pensamentos computacional, lógico e algorítmico, bem como o relacionamento entre eles e a sua aplicação. Além disso, vai ver o conceito e a estrutura de um algoritmo e as principais armadilhas para quem está começando a dar os primeiros passos nessa área.

Lógica e pensamento computacional

O pensamento computacional pode não ser um conceito novo por estar ligado às raízes da própria ciência computacional. Embora já houvesse trabalhos que difundiam a sua relevância para a construção do conhecimento e desenvolvimento do pensamento criativo (PAPERT, 1980), o conceito ganhou reforço nos últimos tempos. Alguns autores como Jeanette Wing, vice-presidente da Microsoft Research e pesquisadora reconhecida na comunidade acadêmica, auxiliaram a analisar de que forma como a ciência computacional analisa o mundo e solucionar problemas seria aplicável a diversos outros contextos (WING, 2006).

Para Wing (2006), o pensamento computacional não está ligado apenas à programação de computadores, mas também à habilidade de pensar logicamente, algoritmicamente e recursivamente. Assim, a ideia não é que as pessoas pensem como um computador, que é limitado ao conhecimento de seu programador, mas que apliquem os conceitos computacionais como estratégia para resolver de forma eficaz problemas nas diversas áreas de conhecimento e no seu cotidiano. Em sua definição, o conceito é baseado em quatro pilares fundamentais, os quais são descritos a seguir.

1. **Decomposição:** segmentar (dividir) problemas complexos em partes mais simples de resolver.
2. **Reconhecimento de padrões:** após a decomposição de um problema, é possível identificar similaridades que algumas dessas partes compartilham. Assim, é possível fazer uso de soluções utilizadas previamente com base em experiências anteriores.
3. **Abstração:** elementos não necessários para resolução do problema são ignorados para que se possa fixar o foco nos detalhes relevantes. Aqui o desafio é selecionar os detalhes que facilitem a compreensão do problema sem perder informações que possam ser importantes para a resolução.

4. **Algoritmo:** o ponto de junção de todos os demais, representado pelo conjunto de instruções claras e finitas necessárias para a solução do problema.

Um dos pilares com grande abrangência em diversas atividades do pensamento computacional, o desenvolvimento do pensamento algorítmico está intrinsecamente ligado ao desenvolvimento da lógica do indivíduo. A seguir, você irá conhecer um pouco mais sobre o seu papel no pensamento computacional.

Evolução da lógica computacional

Embora não seja especificado como um pilar, o pensamento lógico permeia todas as atividades que envolvem o pensamento computacional. A lógica clássica, difundida por Aristóteles e outros pensadores antigos, teve sua origem com o uso de silogismos, uma forma de raciocínio simbolizado por declarações, também chamados de premissas, que descrevem fatos ou conhecimentos de senso comum, e serviram de base para a matemática e para a própria ciência computacional.

Tal representação ordenada e sequencial das premissas descrevem um caminho para resolução do problema. Como exemplo, observe as premissas a seguir.

1. Duas pessoas podem ser consideradas irmãs quando têm ao menos o mesmo pai ou a mesma mãe.
2. Maria é filha de Augusto.
3. João é filho de Augusto.

Considerando que Augusto é mesma pessoa das premissas 2 e 3, podemos inferir que João e Maria são irmãos.

Essas representações também são amplamente utilizadas na matemática, na formulação de teoremas, como: “Se duas grandezas são idênticas a uma terceira, então são idênticas entre si”. Este teorema pode ser formalizado e demonstrado como: “Se $A=C$ e $B=C$, Logo $A=B$ ”. Observe que essa representação matemática é totalmente simbólica: A pode ser qualquer objeto ou valor; mesmo assim, a representação matemática ainda continuará válida.

A lógica é empregada em todas as áreas das ciências, e o conhecimento científico está formalmente representado segundo a lógica, bastante impulsionada por diversos nomes, como George Boole (1815–1864), Gottfried Wilhelm Leibniz (1646–1716), Alfred North Whitehead (1861–1947), Alan Turing (1912–1954), Bertrand Russell (1872–1970) e muitos outros (FIGUEIREDO; LAMB, 2016). Essa ciência deu um salto exponencial a partir do século XX, desde a automatização de verificações matemáticas para a corrida espacial até a atualidade, com a construção de modelos computacionais e simulações para as mais diversas finalidades (modelos climáticos, biomedicina, indústria, esportes de rendimento, mercado financeiro, dentre outras) (FIGUEIREDO; LAMB, 2016).

A lógica e a resolução de problemas

Quando se fala em resolução de problemas, frequentemente são encontrados métodos de raciocínio lógico: o indutivo e o dedutivo. Em alguns momentos, eles podem até parecer sinônimos, mas existem diferenças entre eles.



Fique atento

É importante destacar que o termo **problema** deve ser entendido como qualquer informação, descoberta ou mesmo solução que se deseja encontrar, e não é necessariamente uma situação difícil que precisa de solução. Nesse contexto, esse problema como é tecnicamente denominado **desafio lógico**.

Método indutivo

As premissas ou fatos conduzem a uma “lei de formação” pela repetição de algum padrão de comportamento (LAW, 2008). Veja um exemplo.

- Uma caixa contém 99 bolas vermelhas e uma bola preta.
- 100 pessoas retiraram uma bola da caixa.
- João é uma dessas 100 pessoas.
- Logo, João provavelmente tirou uma bola vermelha.

No método indutivo, deve-se analisar as premissas sobre o problema, buscando padrões que sigam uma lei de formação que possa levar a uma solução. Para esse exemplo, observe que somente uma pessoa irá retirar a bola preta, portanto, conclui-se que a maior probabilidade é de que se retire uma bola vermelha. O método indutivo é utilizado com cautela no meio científico, pois as conclusões obtidas por meio dele podem conduzir a falsas conclusões. A forma do argumento não garante que a conclusão seja verdadeira, mas provavelmente resulta em uma conclusão confiável e próxima à realidade.



Saiba mais

O método indutivo parte de um ponto mais específico para chegar no plano mais genérico, seguindo a sequência:

Observação => formulação das hipóteses => identificação de teorias, enunciados...

Em resumo, caso todas as premissas sejam verdadeiras, a conclusão provavelmente será verdadeira, mas não obrigatoriamente. A conclusão pode chegar a resultados que não estavam nem mesmo implícitos nas premissas.

Método dedutivo

Neste método, as premissas são avaliadas e ordenadas em uma sequência de causa e efeito, de modo que a conclusão obtida seja resultante da sequência completa, emergindo como fato novo extraído ou deduzido a partir das premissas constituintes (LAW, 2008). Veja um exemplo com as seguintes premissas.

- Sempre que chove, o gramado irá molhar.
- Hoje está chovendo.
- Logo, o gramado encontra-se molhado.

O método dedutivo pode precisar de maior quantidade de premissas (todas válidas) para, então, levar a uma conclusão também válida. Esse método é bastante utilizado no método científico, pois busca encontrar a verdade a partir da avaliação de fatos e conhecimentos que possam ser verdadeiramente comprovados, sem qualquer generalização.



Saiba mais

O método dedutivo parte de um plano mais genérico até chegar a um ponto mais específico, seguindo uma sequência:

Formulação do problema > formulação das hipóteses > verificação das hipóteses > avaliação dos resultados

Caso se confirme que as premissas sejam verdadeiras, a conclusão final deve ser verdadeira. A verdade já estava nas premissas, ao menos de forma implícita.

Os computadores usam a lógica da maneira como fazem cálculos, mas isso não é exatamente o mesmo que pensar logicamente no sentido do pensamento computacional. Os próprios computadores devem ser programados (ensinados) para fazer raciocínio lógico por meio de algoritmos. A lógica de computacional é fundamental na criação dos algoritmos, pois tem por objeto de estudo as leis gerais do pensamento e as formas de aplicar essas leis corretamente na investigação da verdade. No próximo capítulo, será abordado o pensamento algorítmico e seus principais constituintes.

Pensamento algorítmico

O pensamento lógico é utilizado cotidianamente sem que as pessoas percebam, por exemplo, ao acordarmos pela manhã e darmos início a uma sequência de etapas até que estejamos prontos para sair de casa, que é quando já estamos utilizando o pensamento lógico e algorítmico sem perceber. Caso essas etapas fossem descritas em ordem de execução como itens de tarefas, estaríamos construindo um algoritmo. Na ciência computacional não é diferente, pois também podemos solucionar problemas utilizando o mesmo pensamento lógico-algorítmico, descrevendo instruções em etapas bem definidas com a finalidade de atingir determinado objetivo.

No momento em que é feita a transição do pensamento lógico para o pensamento algorítmico, é necessário analisar que esses conceitos não são os mesmos. Com a lógica, você tem a compreensão do problema e como ele pode ser subdividido e encontra possíveis soluções para ele. Já no momento em que descrevemos todas as tarefas necessárias e a ordem em que elas devem ser executadas para que o problema seja resolvido, estamos criando um algoritmo.

De acordo com Beecher (2017), essa sequência de passos pode ser utilizada para explicar a um terceiro (seja humano ou uma máquina) como resolver o problema, seguindo tais regras com precisão. Assim como já ocorre com o pensamento lógico, o ser humano possui uma compreensão intuitiva do que é um algoritmo, tão utilizado pela ciência computacional, e desenvolver seu pensamento algorítmico torna-se essencial para o profissional da computação, pois um algoritmo correto é a base final de qualquer solução executada por computador.

Estudiosos da programação como Ada Lovelace (1815—1852), considerada a primeira programadora da história, buscaram formas de comunicar ideias para as máquinas para que elas pudessem compreender e computar por nós, humanos (BEECHER, 2017). Então, concluiu-se que o nosso modelo intuitivo era insuficiente, gerando a necessidade de formular regras de forma clara e inequívoca, como os computadores exigem, evoluindo para estruturas para fornecer instruções aos computadores. Por mais precisos que tenham se tornado, os algoritmos também se tornaram mais complexos, tanto que uma descrição formal deles pode se estender por vários parágrafos. Em nosso caso, a melhor maneira de compreender sua construção é estudar suas estruturas básicas.

Estrutura básica de um algoritmo

A estrutura básica de um algoritmo é formada por vários componentes que determinam como se dará o fluxo de execução da tarefa, desde a inicialização até sua conclusão (MOKARZEL; SOMA, 2008). Para que se possa estruturar o pensamento lógico, é preciso atender a alguns conceitos, os quais são apresentados a seguir.

- **Precisão:** cada etapa em um algoritmo pode ter apenas um significado; caso contrário, a ambiguidade poderá “travar” a sua execução.
- **Ações sequenciais:** as etapas que compõem o processo devem ser realizadas na ordem especificada.
- **Controle dos estados:** deve-se ter controle do *status* de todas as informações gerenciadas pelo sistema em cada momento de execução.
- **Repetição de tarefas:** algumas etapas devem ser executadas múltiplas vezes para evitar que seja preciso reescrevê-las a cada momento.

Seguindo esses requisitos, é possível citar algumas ferramentas algorítmicas que proporcionam a transcrição do pensamento lógico em um conjunto de regras. Veja-as a seguir.

Variáveis e constantes

Auxiliam no armazenamento dos estados e informações durante a execução do processo. Tal estrutura pode armazenar temporariamente informações adicionais que serão processadas pelas instruções, e seus valores podem sofrer variações durante a execução do algoritmo, por isso são conhecidas como “variáveis”. Já as “constantes” são as que guardam valores que devem permanecer inalterados durante toda a execução do algoritmo (por exemplo, o valor da constante π , cujo valor aproximado é 3,141593, calculada pela razão entre a medida da circunferência de um círculo — $2\pi r$ — e o seu diâmetro — $2r$, na geometria).



Exemplo

Veja a seguir alguns exemplos de variáveis.

- Inteiro: números reais inteiros. Exemplos: 10, 9, 8.
- Real: números reais com valor decimal. Exemplos: 3.5, 4.0, dízimas periódicas.
- Caractere: letra, número ou símbolo. Exemplos: 'A', 'B', '1B', '2'.
- Texto: letras. Exemplo: “texto de exemplo”.
- Lógico: lógicos. Exemplo: “verdadeiro e falso”.

Para utilizar variáveis, inicialmente é preciso criá-las. Em programação, o ato de criar o espaço na memória do computador para armazenamento é conhecido como “declaração de variável”, em que é possível, por exemplo, colocar etiquetas em pequenas caixas para identificar o que pode ser armazenado em cada uma delas. Após identificada, a caixa pode receber apenas itens do mesmo tipo, como, por exemplo, uma caixa com a etiqueta “papéis” que só pode armazenar papel. Esse é um conceito importante, em que cada variável declarada deve ter somente um nome (identificador) e um tipo, que é o valor que pode ser armazenado nela (numérico, textual, lógico, entre outros). Determinadas linguagens de programação permitem a troca do tipo de informação durante a execução do programa, já outras são mais restritas e só permitem armazenar o tipo de dado identificado no momento da sua criação.

Operadores

São utilizados para realizar operações diversas, dependendo de sua categoria. Veja cada um deles a seguir.

- **Aritméticos:** essenciais para determinadas tarefas, como a realização do cálculo de uma equação. Tarefas como adição, subtração, multiplicação e divisão.
- **Atribuição:** utilizados no armazenamento de valores em variáveis.
- **Relacionais:** utilizados para comparação entre valores diversos. Alguns exemplos são: “maior que”, “menor que”, “igualdade”, entre outros.
- **Lógicos:** empregados na conexão entre duas condições ou para negar uma expressão. Os operadores lógicos mais usados são o “e” (AND), o “ou” (OR) e o “não” (NOT).

Veja no Quadro 1 os operadores relacionais.

Quadro 1. Operadores relacionais

Operador	Comparação
==	Igual a
!=	Diferente de
<	Menor que
>	Maior que
<=	Menor que ou igual a
>=	Maior que ou igual a

Fonte: Adaptado de Noronha (2019).

Condicionais

Em determinados momentos, durante a execução das tarefas, é preciso tomar algum tipo de decisão. Por exemplo: deve-se armazenar um número somente se o resultado de uma soma for maior que um valor de referência; caso contrário, o número deve ser descartado. As estruturas condicionais permitem a execução de determinadas instruções considerando situações (condições) específicas que precisam ser satisfeitas.

Uma das instruções condicionais mais utilizadas é a estrutura **Se—Então**. Na prática, ela é utilizada quando é preciso tomar uma decisão de realizar ou não determinada ação. Por exemplo: na situação “O aluno está aprovado?”, é necessária uma condição que se possa comparar e, então, dar a resposta. Pode-se, então, considerar que o aluno deve ter média maior ou igual a 7 para aprovação. Logo, a estrutura condicional seria:

“Caso o aluno tenha média igual ou maior que 7, então ele estará aprovado.”

Observe que a avaliação é sempre “Se o condicional é verdadeiro, então...”. No entanto, nem sempre os alunos estarão aprovados, então também pode-se ter uma resposta não positiva. Assim, completa-se a estrutura com a cláusula **Senão**, ficando no formato **Se—Então—Senão**. Dessa forma, pode-se completar o teste condicional, que ficaria assim:

“Caso o aluno tenha média igual ou maior que 7, então ele
estará aprovado, senão ele estará reprovado.”

Seu formato algorítmico é:

Se(condição)

Então "ação executada caso o teste seja verdadeiro"

Senão "ação executada caso o teste seja falso"

FimSe



Fique atento

Grande parte das linguagens de programação utiliza a língua inglesa como padrão para definição de sua sintaxe. Dessa forma, o comando em pseudocódigo **Se—Então—Senão** é transcrito para `if-then-else`, e em boa parte das linguagens atuais a cláusula `then` está implícita.

Logo, o mesmo exemplo na linguagem de programação PHP (considerando os caracteres de limitação de código próprios da linguagem) seria transcrito da seguinte forma:

```
if(condição){
    "ação executada caso o teste seja verdadeiro"
} else {
    "ação executada caso o teste seja falso"
}
```

Repetições

Permitem a execução de uma mesma instrução várias vezes até que uma condição seja alcançada. Utilizando essa ferramenta, pode-se tanto repetir uma ação em um número exato de vezes (p. ex.: bloquear um acesso após três tentativas) como repetir determinadas instruções até uma condição ser atendida (p. ex.: estude até entender a lição, independentemente do tempo que levar). Qualquer uma destas formas exige uma condição para que a repetição cesse, seja por terminar o número de iterações especificado ou por chegar ao resultado desejado.

No caso de repetições em número predeterminado, tem-se a estrutura **Para**. Com ela é possível, por exemplo, ter a situação “Repita de 1 até 5 a ação: escolha aleatoriamente uma letra do alfabeto e escreva na tela para o usuário”.

Sua estrutura em pseudocódigo pode ser representada assim:

```
Para variável de (valor de início) ate (valor do fim) faça
    instruções
FimPara.
```

No caso de uma repetição sem quantidade definida, ou seja, até que algo aconteça, pode-se utilizar a estrutura **Enquanto**, que significa que, como o próprio nome diz, enquanto houver a condição de teste, ele continuará repetindo. Essa estrutura utiliza um condicional para avaliar se ele deve ser executado, ou seja, só é possível entrar na estrutura de repetição **Enquanto** se a condição teste for verdadeira, senão ela não irá entrar no laço de repetição. Sua estrutura em pseudocódigo pode ser representada assim:

```
Enquanto(condição) faça
    instruções
FimEnquanto
```



Saiba mais

Os comandos de repetição **para** (**for**) e **enquanto** (**while**) também seguem como padrão a língua inglesa, e suas estruturas transcritas para linguagem de programação PHP são:

```
for (expr1; expr2; expr3)
    instrução a ser repetida
```

onde cada uma das expressões auxilia a controlar o número de repetições que devem ser executadas.

```
while (expr)

    instrução a ser repetida
```

O propósito da declaração `while` é simples: enquanto a condição `expr` for avaliada como `TRUE`, a repetição é executada. Entretanto, se a expressão `expr` for avaliada como `FALSE` desde o início, as instruções não serão executadas nenhuma vez.

Existe uma variação da estrutura **enquanto**, o `do-while`, e sua diferença em relação ao anterior é o momento em que o teste condicional é realizado. Neste caso, as instruções são realizadas ao menos uma vez e, então, o condicional em `expr` é executado avaliando se ela deve continuar repetindo a instrução. Na sintaxe, o teste condicional é movido para o final da estrutura:

```
do {

    instrução a ser repetida

} while (expr);
```

Exemplificando um algoritmo em linguagem natural

Pode-se utilizar várias ferramentas para representar ou codificar um algoritmo, e para isso foram desenvolvidas ao longo de décadas diversas linguagens de programação. É importante lembrar que para desenvolvermos o pensamento algorítmico não é preciso aprender a programar, mas conhecer essas estruturas podem auxiliar no seu desenvolvimento lógico-algorítmico.

Como exemplo, vamos utilizar um algoritmo sugerido por Beecher (2017), que parte de um processo para criação de um jogo. Para exemplificarmos, será descrita em uma linguagem natural uma estrutura informal para escrita mais próxima à fala humana, mas que segue as convenções utilizadas nas linguagens de programação formais. Ou seja, a estrutura criada com pseudocódigo pode tranquilamente ser traduzida para qualquer linguagem de programação real, apenas substituindo as instruções informais para as respectivas da linguagem.

Considere a seguinte lista de regras para o conhecido “jogo da velha”.

1. iniciar o jogo
2. iniciar uma repetição:
 3. pedir ao jogador para escolher um quadrado

4. se o quadrado escolhido não estiver ocupado, coloque o símbolo do jogador naquela posição
5. verifique se após a jogada foi formada uma linha
6. se uma linha for formada, o jogador vence e o jogo encerra
7. se uma linha não foi formada e nenhum quadrado estiver disponível, então o jogo está empatado
8. mude para outro jogador
9. encerrar repetição se o jogador estiver ganho ou empatado
10. exibir a mensagem 'Fim de jogo'

O que podemos inferir desse roteiro de instruções:

- Quando esse algoritmo é executado, o computador passa por todas as linhas, uma de cada vez (este é um exemplo de sequência).
- A linha 1 inicializa uma variável chamada “início”, logo que o jogo começa, e “início” recebe o valor verdadeiro (armazenamento de estado).
- A linha 2 configura o ponto inicial de uma repetição. Todas as instruções que devem ser repetidas são recuadas para deixar mais claro o que está sendo executado.
- A linha 3 solicita uma ação do jogador. A escolha é registrada.
- A linha 4 faz uma seleção com base na escolha do jogador.
- As linhas 6 e 7 fazem comparações condicionais, alterando o valor da variável do jogo sob certas condições.
- A linha 9 é o fim da repetição. É feita a escolha de iniciar ou não a repetição novamente, dependendo da condição associada neste ponto. Existem duas possibilidades:
 - Verifica que o jogador não conseguiu criar uma linha com três símbolos iguais. Nesse caso, a execução volta para a linha 3.
 - Em algum momento, o jogo foi ganho ou empatado. Em qualquer dos casos, a repetição encerra e a execução continua na linha 10, exibindo em seguida a mensagem 'Fim de jogo'.

Dificuldades recorrentes com lógica e algoritmos

Por mais simples e rotineiro que os conceitos de pensamento lógico e pensamento algorítmico possam parecer, nos primeiros contatos com a estruturação da solução de um problema podem gerar alguns erros ou até suposições

incorretas. Isso se dá em parte pela forma com que lidamos com essas soluções, em que normalmente executamos as tarefas sem nos darmos conta.

Segundo Beecher (2017), é possível que uma pessoa que execute a mesma tarefa durante anos tenha dificuldade para ensinar outra pessoa a realizar essa tarefa. E, ao criarmos um algoritmo para ser executado por um computador, o problema é maior ainda, pois a falta de uma simples instrução é capaz de parar o funcionamento de um sistema complexo.

Necessidade de clareza e meticulosidade

Clareza é uma palavra-chave no momento de descrever um algoritmo, pois esta é uma tarefa que exige que o autor seja muito criterioso na descrição e ordenação das ações. Como já mencionado, a falta de uma simples regra pode ocasionar graves problemas de execução ou respostas erradas após o processamento (MOKARZEL; SOMA, 2008).

Uma máquina não racionaliza as tarefas que está recebendo, apenas executa o que foi descrito e da forma que foi descrita. Imagine que você esteja explicando para um autômato (um robô, por exemplo) que pegue um objeto dentro de uma caixa e, caso ela esteja fechada e o desenvolvedor não tenha explicitado que “se a caixa estiver fechada, abra a caixa”, ele poderá não saber o que fazer e ficará parado. Afinal, ele não pensa, apenas executa o que foi instruído. Também não é possível exigir que ele tenha bom senso, nem que tente interpretar o que o autor estava tentando explicar, mesmo que a solução seja óbvia.

Resumidamente, qualquer instrução que não esteja explícita ou ambígua não será executada. Da mesma forma, caso ele receba um comando que irá gerar erro, como tentar atravessar uma parede, ele tentará fazê-lo de qualquer forma. Esta é uma das principais dificuldades para quem começa a desenvolver seu pensamento computacional e, quanto mais desenvolvermos a estruturação do raciocínio, cada vez menos esses erros acontecem.

Uso incorreto de operadores lógicos

O uso incorreto de operadores lógicos é bem comum no cotidiano, e algo aparentemente tão simples como o uso das palavras “e”, “ou”, “não” e “então” pode nos surpreender, como, por exemplo, a instrução “Todos os alunos que nasceram em março e abril podem realizar suas matrículas no semestre”.

Lembre-se que o computador não usará o bom senso e, mesmo que pareça lógico, o computador irá gerar um erro e não permitirá que nenhum

aluno se matricule. Mas por quê? Na linguagem natural, ou seja, na forma em que falamos, pode-se vez ou outra utilizar incorretamente os conectivos “e” e “ou”, e mesmo assim a outra pessoa irá interpretar o que estávamos tentando dizer. Mas lembre-se que o computador não interpreta, apenas executa (MADEIRA, 2020).

O operador “e” exige que ambas as condições sejam verdadeiras para que a ação possa ser executada, considerando como primeira condição “ter nascido em março”, como segunda condição “ter nascido em abril” e como ação a ser executada “realizar a matrícula”. Na forma em que está escrito, o aluno precisaria ter nascido ao mesmo tempo em março e em abril (ambas necessitam ser verdadeiras). Para um humano isso é intuitivo, mas a máquina não saberá o que fazer e irá falhar. Então, para corrigir o teste, visto que se quer permitir a matrícula apenas para os alunos nascidos em março e abril, deve-se substituir o operador “e” pelo operador “ou”, e então qualquer aluno nascido no mês de março ou no mês de abril poderia se matricular (MOKARZEL; SOMA, 2008).



Fique atento

Operadores lógicos são utilizados na avaliação entre duas ou mais expressões booleanas (lógicas), em que o resultado dessa avaliação retorna verdadeiro (TRUE) ou falso (FALSE). Os operadores booleanos mais utilizados são:

E (em programação AND):

Apenas quando todas as expressões testadas são verdadeiras, o resultado será verdadeiro. Para o condicional `expr1 and expr2`:

Expr1	Expr2	Resposta do condicional
True	True	True
True	False	False
False	True	False
False	False	False

OU (em programação OR):

Quando ao menos uma das expressões testadas for verdadeira, o resultado será verdadeiro. Para o condicional `expr1 or expr2`:

Expr1	Expr2	Resposta do condicional
True	True	True
True	False	True
False	True	True
False	False	false

Essas comparações utilizando operadores lógicos são frequentemente utilizadas em estruturas condicionais, indicando que algo deva ser realizado caso a condição seja satisfeita. A estrutura **Se—Então** é um exemplo claro dessa aplicação, e somente **Se** algo acontecer (aluno ter nascido em março ou em abril) a instrução pode ser realizada (podem realizar a matrícula).

Testes condicionais incompletos ou com lacunas não tratadas

Voltando ao último exemplo, pode-se considerar que todos os alunos nascidos em março ou em abril podem iniciar seu processo de matrícula, o que está correto em parte. Mas o que acontece com o restante dos alunos que não nasceram nestes meses? É claro que eles devem aguardar o período de acordo com seu mês de nascimento para a matrícula, mas lembre-se que a máquina não possui bom senso.

Assim como a ação que deve ser realizada quando o teste lógico resultar verdadeiro (o aluno que nasceu em março ou abril), também deve ser descrito o que deve ser feito quando o teste resultar em falso (para todos os demais meses). Neste caso, poderia ser apresentada uma mensagem informando ao aluno que ele deve aguardar, por exemplo. Pode-se completar nossa estrutura condicional “se—então” incluindo uma cláusula “senão”, em que esta seria executada somente quando a condição testada for falsa (MOKARZEL; SOMA, 2008).

Referências

- BEECHER, K. *Computational thinking: a beginner's guide to problem-solving and programming*. Swindon: BCS Learning & Development, 2017. 288 p.
- FIGUEIREDO, C. M. H.; LAMB, L. C. Teoria da Computação: uma Introdução à Complexidade e à Lógica Computacional. In: JORNADA DE ATUALIZAÇÃO EM INFORMÁTICA, 34., 2015, Porto Alegre. *Anais [...]*. Porto Alegre: Sociedade Brasileira de Computação, 2015, p. 10–67. Disponível em: <https://sol.sbc.org.br/livros/index.php/sbc/catalog/download/6/6/19-1?inline=1>. Acesso em: 18 dez. 2020.
- MOKARZEL, F.; SOMA, N. *Introdução à ciência da computação*. Rio de Janeiro: Elsevier, 2008. 429 p.
- NORONHA, C. Lógica e Algoritmos de Programação. *Medium*, San Francisco, 13 fev. 2019. Disponível em: <https://medium.com/@caio.cnoronha/l%C3%B3gica-e-algoritmos-de-programa%C3%A7%C3%A3o-47b4a872b815>. Acesso em: 18 dez. 2020.
- PAPERT, S. *Mindstorms: children, computers, and powerful ideas*. New York: Basic Books, 1980. 230 p.
- WING, J. M. Computational thinking. *Communications of the ACM*, New York, v. 49, n. 3, p. 33–35, Mar. 2006.

Leituras recomendadas

- MADEIRA, C. Introdução ao Pensamento Computacional com Scratch. In: CONGRESSO SOBRE TECNOLOGIAS NA EDUCAÇÃO, 2., 2017, Mamanguape. *Anais [...]*. Mamanguape: Universidade Federal da Paraíba, 2017. p. 725–730. Disponível em: http://ceur-ws.org/Vol-1877/CtrlE2017_MC_4.pdf. Acesso em: 24 nov. 2020.
- OLIVEIRA, C. M. et al. Utilização de Desafios para o Desenvolvimento do Pensamento Computacional no Ensino Superior: Um Relato de Experiência. In: SIMPÓSIO BRASILEIRO DE INFORMÁTICA NA EDUCAÇÃO, 30., 2019, Brasília. *Anais [...]*. Brasília: Sociedade Brasileira de Computação, 2019. p. 2005–2014. Disponível em: <https://www.br-ie.org/pub/index.php/sbie/article/view/8934>. Acesso em: 18 dez. 2020.
- PAULA, B. H.; VALENTE, J. A.; BURN, A. O uso de jogos digitais para o desenvolvimento do currículo para a Educação Computacional na Inglaterra. *Currículo sem Fronteiras*, [S. l.], v. 14, n. 3, p. 46–71, 2014. Disponível em: <http://www.curriculosemfronteiras.org/vol14iss3articles/paula-valente-burn.htm>. Acesso em: 18 dez. 2020.
- RILEY, D. D.; HUNT, K. A. *Computational thinking for the modern problem solver*. Boca Raton: CRC Press, 2014. 406 p. (Chapman & Hall/CRC Textbooks in Computing).



Fique atento

Os links para sites da web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integridade das informações referidas em tais links.

Conteúdo:

